

LLM-Driven Pick and Place with a 4-DOF Manipulator

Valentin Gaucher*

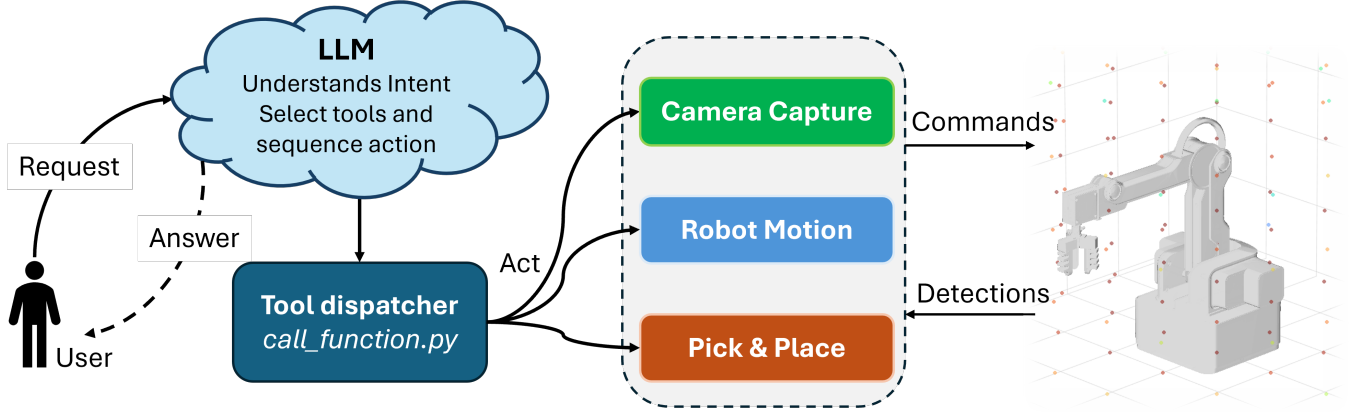


Fig. 1: Architecture of the LLM-driven pick-and-place system. A user send natural language commands to the LLM (Gemini here) which select and uses tools through the dispatcher function. Dashed lines indicate feedback loops.

Abstract—This report describes the design, implementation, and experimental validation of a large language model (LLM) driven pick-and-place system employing a Dobot Magician 4-DOF robotic manipulator, a fixed overhead USB camera, and the Google Gemini API. The system addresses the problem of natural-language-commanded block manipulation: given a workspace populated with colored blocks, a user issues text instructions such as “place blue on top of green” or “move red to the left of blue,” and the LLM autonomously orchestrates the perception, planning, and execution pipeline through function calling. The architecture is organized around three subsystems: (1) a vision module that captures the scene, detects colored blocks via HSV segmentation, and records their pixel centroids, (2) a calibrated coordinate-transformation layer that maps pixel positions to robot-frame coordinates through a precomputed affine matrix and (3) a pick-and-place execution module that computes safe approach heights, activates the suction gripper, and sequences the motions for stacking or side-by-side placement. The Gemini model acts as the central controller, receiving tool definitions as function declarations and selecting the appropriate tool calls in response to user queries. Experimental results demonstrate that the system correctly identifies blocks, resolves spatial references, and executes multi-step pick-and-place operations with calibrated heights and offsets. Video of the experiments is available here: <https://youtube.com/shorts/hCylxk1QtvU?si=0s2ebL9ZZZBJhvxS>

I. INTRODUCTION

The integration of large language models (LLMs) with robotic manipulation is an area of growing interest in the robotics community. Recent work has demonstrated that pre-trained language models can serve as high-level task planners when augmented with tool-use capabilities, enabling robots to interpret and execute natural-language commands without hand-coded state machines [1], [2].

The objective of this project is to design a system in which a Dobot Magician manipulator picks and places colored blocks on a table in response to natural-language instructions issued by a human operator. The system must support three classes of spatial operations: **Stacking**: picking a specified block and placing it on top of another. **Lateral placement**: picking a block and placing it to the left, right, front, or back of another block. **Scene understanding**: capturing the current workspace, detecting all visible blocks, and reporting their identities and locations to the user. The high level architecture is reported Figure 1.

Unlike the maze-solving project described in a prior Project 1, which relied on a deterministic vision pipeline with hard-coded stage transitions, the present system delegates the sequencing of perception and action entirely to an LLM. The Gemini model receives a set of callable tools (camera capture, robot motion primitives, and a composite pick-and-place function) and autonomously decides which tools to invoke and in what order based on the user’s natural-language request.

The remainder of this report is organized as follows. Section II presents the background on LLM tool use and the coordinate transformations reused from the calibration stage. Section III describes the system architecture and the responsibilities of each module. Section IV details the implementation of each component. Section V reports the experimental procedure. Section VI presents the results. Section VII discusses limitations and possible improvements. Section VIII concludes.

II. BACKGROUND

A. LLM Function Calling

Modern LLMs such as Google Gemini [3] support *function calling* (also called tool use), a mechanism by which the

This work is a project for ASU Course RAS-545.

PhD Student, Arizona State University, Mesa, AZ, 85212, USA.

Email: {vgaucher}@asu.edu.

model can emit structured invocations of externally defined functions rather than generating free-form text. The developer supplies a set of `FunctionDeclaration` schemas (each specifying a function name, description, and parameter types) and the model selects which function(s) to call based on the conversation context. The runtime dispatches the call, returns the result to the model as a tool response, and the model may then issue further calls or produce a final natural-language reply. This transforms the LLM from a text generator into a *reasoning agent* capable of closed-loop interaction with external systems. In the present project, the external system is the Dobot Magician and its associated camera.

B. Affine Pixel-to-Robot Transformation

The coordinate mapping between the overhead camera and the robot workspace is identical to the one described in the prior calibration project 1. A set of N manually collected correspondences $\{(\mathbf{u}_i, \mathbf{X}_i)\}_{i=1}^N$ between pixel locations $\mathbf{u}_i = (u_i, v_i)$ and robot positions $\mathbf{X}_i = (X_i, Y_i)$ are used to recover a 2×3 affine matrix $\mathbf{M}$ by solving

$$\mathbf{M}^* = \arg \min_{\mathbf{M}} \sum_{i=1}^N \|\mathbf{M}\tilde{\mathbf{u}}_i - \mathbf{X}_i\|_2^2, \quad (1)$$

where $\tilde{\mathbf{u}}_i = (u_i, v_i, 1)^\top$. The recovered matrix is stored in the configuration file and applied at runtime to convert detected block centroids from pixel space to robot coordinates:

$$\begin{pmatrix} X \\ Y \end{pmatrix} = \mathbf{M} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}. \quad (2)$$

III. SYSTEM ARCHITECTURE

The system consists of five principal components connected through the Gemini function-calling loop. The architecture is summarized below.

LLM Controller

(`LLM_ROBOT.py`). The main entry point. It initializes the Gemini client, registers all tool schemas, manages the conversation history, and dispatches tool calls via the `call_function` module.

Tool Dispatcher

(`call_function.py`). Maps function names emitted by the LLM to their Python implementations. For file-related tools, it injects the working directory; for robot and camera tools, it forwards the arguments directly.

Camera Capture Module

(`Camera_Capture_Tools.py`). Opens the overhead camera, runs a real-time block detection loop using HSV color segmentation, annotates each detected block with a label (e.g., `red1`, `blue2`), and after a configurable countdown saves the annotated frame and a JSON file containing the labels and pixel centroids.

Robot Motion Module

(`Robot_Motion_Tools.py`). Provides low-level motion primitives: connecting to the Dobot, homing, moving to Cartesian poses, moving to pixel-derived positions via the affine transform, and controlling the suction gripper.

Pick-and-Place Module

(`Pick_Place_Tool.py`). Implements the high-level pick-and-place behavior. Given a source block label, a target block label, a placement type (`on_top` or `beside`), and optionally a direction (`front`, `back`, `left`, `right`), it reads the detection JSON, computes the motion sequence with calibrated heights and offsets, and executes the full pick-place cycle.

IV. METHOD

A. Block Detection via HSV Segmentation

The vision module detects colored blocks on a dark background by converting each camera frame to the HSV color space and applying per-color masks for blue, green, yellow, and red. Each color is defined by one or more $(H_{\min}, S_{\min}, V_{\min})$ – $(H_{\max}, S_{\max}, V_{\max})$ intervals. Red requires two intervals because its hue wraps around in the HSV cylinder.

The binary masks undergo morphological opening and closing with a 5×5 kernel to remove noise. Contours are extracted and filtered by area (minimum threshold $A_{\min} = 500$ pixels²). For each valid contour, the bounding rectangle and image moments are computed, yielding the centroid (c_x, c_y) . Blocks are labeled sequentially within each color class (e.g., `green1`, `green2`, `green3`).

B. Height Computation for Pick and Place

All vertical positions are computed relative to the table contact height z_{table} and the physical block dimensions. The key constants are:

- $z_{\text{table}} = -45$ mm: Z coordinate at table surface contact.
- $z_{\text{above}} = 100$ mm: safe travel height above the workspace.
- $h_{\text{block}} = 40$ mm: physical block thickness.
- $l_{\text{block}} = 20$ mm: physical block side length.
- $\delta_{\text{stack}} = 10$ mm: clearance added when stacking.
- $\delta_{\text{side}} = 10$ mm: XY gap when placing beside.

For the **pick** sequence, the suction gripper descends to the pickup height:

$$z_{\text{pick}} = z_{\text{table}} + h_{\text{block}}. \quad (3)$$

For **stacking** (`on_top`), the place height accounts for two block thicknesses plus clearance:

$$z_{\text{place}}^{\text{stack}} = z_{\text{table}} + 2h_{\text{block}} + \delta_{\text{stack}}. \quad (4)$$

For **lateral placement** (`beside`), the block is placed on the table at single-block release height, and the XY position is offset from the target by

$$d_{\text{offset}} = l_{\text{block}} + \delta_{\text{side}} \quad (5)$$

in the requested direction, where directions are mapped to robot axes.

C. Pick-and-Place Motion Sequence

The complete motion sequence for a pick-and-place operation is:

- 1) Move above source block at z_{above} .
- 2) Descend to z_{pick} at source pixel coordinates.
- 3) Activate suction.
- 4) Ascend to z_{above} .
- 5) Move above target position at z_{above} .
- 6) Descend to z_{place} .
- 7) Deactivate suction.
- 8) Ascend to z_{above} .

For `on_top` placement, steps 5–6 use the target block’s pixel coordinates transformed through the affine matrix. For `beside` placement, the target pixel coordinates are first converted to robot coordinates via Equation (2), the offset from Equation (5) is applied, and the resulting robot-frame position is commanded directly.

D. System Prompt Design

The system prompt instructs the Gemini model on its role and operational constraints. The key directives encoded in the prompt are:

- 1) The model controls a Dobot robotic arm with a suction gripper and an overhead camera.
- 2) All physical actions must be performed through tool calls; the model must never claim an action occurred without invoking the corresponding tool.
- 3) When the user requests scene understanding, the model should first call `move_to_home()` to clear the field of view, then call `capture_scene_with_detection()` to capture and detect blocks.
- 4) After capture, the model must summarize all detected blocks with their labels and pixel coordinates, then prompt the user for the next action.
- 5) For pick-and-place requests, the model must verify that all required parameters are specified (source label, target label, placement type, and direction if applicable). If any are missing, the model must ask for clarification before proceeding.
- 6) The model must only reference block labels from the most recent capture and must never invent or guess labels.

E. Conversation Loop

The main loop in `LLM_ROBOT.py` implements a multi-turn conversation with interleaved tool execution. For each user message, the system allows up to 20 iterations of model generation and tool dispatch. Within each iteration:

- If the model produces text, it is printed as the response.
- If the model emits function calls, each call is dispatched through `call_function()`, and the result is appended to the conversation history as a tool response.
- The loop continues until the model produces a text response with no further tool calls, at which point the system awaits the next user input.

V. EXPERIMENTS

A. Hardware Setup

The experimental platform consists of a Dobot Magician manipulator connected over USB to a host computer on the serial port `/dev/ttyACM0`. A USB overhead camera captures frames at a resolution of 640×480 pixels using MJPG encoding. Colored blocks (blue, green, yellow, red) are placed within the reachable workspace.

B. Calibration

The affine transformation matrix M was computed from manually collected pixel-to-robot correspondences using `cv2.estimateAffine2D`, following the same procedure as the prior calibration project. The matrix used in all experiments is:

$$M = \begin{pmatrix} 6.007 \times 10^{-3} & -4.842 \times 10^{-1} & 3.807 \times 10^2 \\ -4.691 \times 10^{-1} & 3.750 \times 10^{-3} & 1.553 \times 10^2 \end{pmatrix}. \quad (6)$$

C. Test Scenario

The workspace was populated with 4 blocks detected as shown in Fig 2. The system was run interactively using `python3 LLM_ROBOT.py "Understand the scene"` to first capture and identify the blocks, followed by pick-and-place commands.

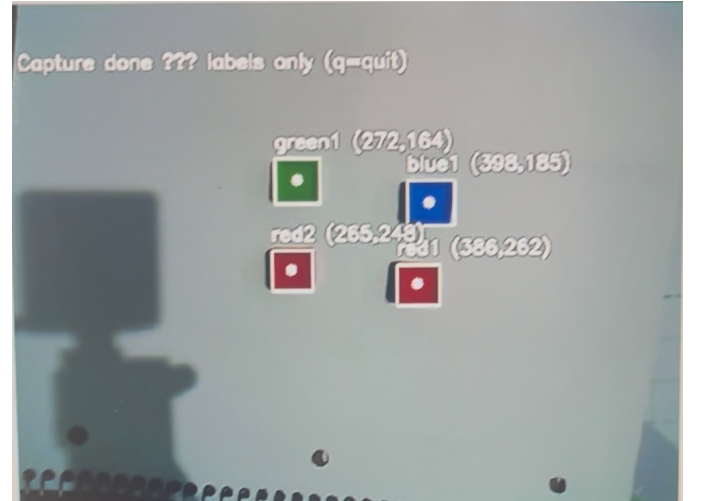


Fig. 2: Camera output from the user request "Understand the scene".

VI. RESULTS

A. Scene Understanding

When prompted with "Understand the scene," the LLM correctly executed the following tool-call sequence:

- 1) `move_to_home()`: moved the arm out of the camera field of view.
- 2) `capture_scene_with_detection()`: open the camera, ran a 5-second countdown, detected five blocks, and saved the annotated image and JSON file.

The model then produced a text summary listing all four blocks with their pixel coordinates and asked the user which block to move and where, matching the expected behavior specified in the system prompt.

B. Pick and Place Operations

The following operations were tested successfully as shown here <https://youtube.com/shorts/hCylxk1QtvU?si=0s2ebL9ZZZBJhxvS>:

Stacking. The command “place blue1 on top of green1” resulted in the LLM calling `pick_and_place_block` with parameters `source_label="blue1"`, `target_label="green1"`, `placement_type="on_top"`. The robot picked the blue block from its detected position, traveled at the safe height, descended onto the green block at the computed stack height ($z_{\text{table}} + 2h_{\text{block}} + \delta_{\text{stack}} = -45 + 80 + 10 = 45$ mm), released suction, and retracted.

Lateral placement. The command “place red1 to the right of green1” resulted in the LLM calling `pick_and_place_block` with `placement_type="beside"` and `direction="right"`. The robot converted the target pixel coordinates to robot-frame via the affine matrix, applied the offset $d_{\text{offset}} = 20 + 10 = 30$ mm in the +Y direction, and placed the red block at the computed position.

C. Error Handling

When a user requested a nonexistent block label (e.g., “move yellow1 on top of blue1”), the `pick_and_place_block` function returned an error response including the list of available labels. The LLM relayed this information to the user and prompted for a corrected command. When a user issued an ambiguous command (e.g., “stack the blocks”), the LLM correctly asked for clarification about which blocks and in what order, rather than guessing.

VII. DISCUSSION

A. Advantages of the LLM-Based Approach

The principal advantage of delegating task sequencing to an LLM is flexibility. Adding a new operation requires only defining a new tool schema and its implementation; the LLM will discover when to use it based on user intent without modifying the control flow. The conversational interface also provides natural error recovery: when information is missing or ambiguous, the model can ask clarifying questions rather than failing silently.

B. Limitations

Several limitations of the current implementation are noted: **API rate limits.** The free tier of the Gemini API permits only 20 requests per day per key (with up to 6 keys). This severely constrains iterative development and testing. **Static scene assumption.** The system captures the scene once and operates on the resulting detection. If blocks are displaced during

manipulation (e.g., by imprecise placement or collision), the stored detections become stale. **No collision checking.** The pick-and-place module does not verify whether the computed trajectory would collide with other blocks in the workspace. **HSV sensitivity.** The color detection depends on lighting conditions. Changes in ambient illumination can cause missed or spurious detections. **Single-layer stacking.** The current height model supports only two-block stacks. Extending to arbitrary stack heights would require tracking the current stack count per position.

C. Future Work

Natural extensions include: (1) re-capturing the scene after each manipulation to maintain an up-to-date world model; (2) implementing multi-block stacking with a stack-height tracker; (3) adding collision avoidance by checking block positions before computing trajectories; and (4) migrating to a vision-language model that can process the camera image directly rather than relying on the intermediate JSON representation.

VIII. CONCLUSION

This project demonstrated that a large language model can serve as an effective high-level controller for a robotic pick-and-place system when provided with well-defined tool interfaces. The Gemini model, given a system prompt describing its role and a set of function declarations for camera capture, robot motion, and pick-and-place execution, was able to autonomously interpret natural-language commands, orchestrate the appropriate tool calls, handle missing or ambiguous information through clarifying questions, and successfully execute block manipulation tasks including stacking and lateral placement. The modular architecture (separating the LLM controller, vision module, motion primitives, and high-level pick-and-place logic) facilitated development and debugging. The calibrated affine transformation from the prior project provided accurate pixel-to-robot coordinate mapping, enabling precise block positioning.

REFERENCES

- 1 M. Ahn *et al.*, “Do As I Can, Not As I Say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- 2 S. Vemprala, R. Bonatti, A. Buckner, and A. Kapoor, “ChatGPT for robotics: Design principles and model abilities,” *IEEE Access*, vol. 12, pp. 55682–55696, 2024.
- 3 Google DeepMind, “Gemini: A family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2024.
- 4 J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. Pearson, 2018.
- 5 G. Bradski, “The OpenCV library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.